PROJECT AND TEAM INFORMATION

Project Title

Custom version control system (mini Git)

Student / Team Information

Team Name: *Team ID:*	*CODE RUSHERS* DSCPP-III-2026-T358
Team member 1 (Team Lead) *Student ID:*	***Singh, Jasmeet -*** 
Team member 2 *Student ID: 260211526*	Devraj Raushan - 260211526 260211526@geu.ac.in 

Faculty: *prof Dr.Deepak arsani*
Email: *deepakarsani.cse@geu.ac.in*

PROJECT DESCRIPTION

This project implements a simplified version control system, inspired by Git, to demonstrate core data structures and object-oriented design principles used in real-world distributed VCS tools. The system models Git's internal architecture using content-addressable storage, where every file and directory snapshot is uniquely identified through cryptographic hashing (SHA-1), ensuring automatic deduplication and data integrity.

The design follows an object-oriented hierarchy comprising four core entities — Blob (file content), Tree (directory structure), Commit (snapshot metadata with parent references), and Repository (the orchestrating controller). Commit history is modeled as a Directed Acyclic Graph (DAG), enabling support for branching and merge commits with multiple parents, unlike a simple linear or tree structure.

The centerpiece of the project is a custom-built diff algorithm, implemented using dynamic programming (Longest Common Subsequence-based or Myers' diff), which efficiently computes line-level differences between file versions. This is extended into a three-way merge mechanism that identifies the lowest common ancestor between diverging branches and detects conflicting changes, replicating Git's core merge behavior.

Core functionalities implemented include repository initialization, staging, committing, branching, checkout, logging, diffing, and merging — all accessible via a command-line interface. The project demonstrates practical applications of hashing, trees, graphs, and dynamic programming within a cohesive, real-world software system.

STATE OF ART / CURRENT SOLUTIONS

Version control today is dominated by Git, created by Linus Torvalds in 2005, which has become the industry standard for tracking source code changes. Git models a repository as a content-addressable object database: every file (blob), directory snapshot (tree), and commit is hashed (SHA-1, transitioning to SHA-256) and stored immutably, enabling deduplication, integrity verification, and efficient history traversal. Commit history forms a Directed Acyclic Graph (DAG), where each commit references one or more parent commits — supporting linear development as well as branching and merging.

Existing solutions built on this model include GitHub, GitLab, and Bitbucket, which add collaboration layers (pull requests, code review, CI/CD) on top of Git's core engine. Git itself uses advanced optimizations in production — packfiles for compressed storage, delta encoding between similar objects, and the Myers diff algorithm for efficient line-by-line comparison between versions.

Alternative VCS tools like Mercurial and Subversion (SVN) use similar or centralized models but have lost adoption due to Git's superior branching model and distributed architecture.

While Git is highly optimized and production-grade, its internal mechanics (hashing, DAG-based history, diffing) are rarely implemented from scratch by learners, motivating this project's from-first-principles approach.

PROJECT GOALS AND MILESTONES

General Goal: To design and implement a simplified version control system (Mini-Git) that replicates the core internals of Git — content-addressable storage, commit history tracking, branching, and file diffing — using fundamental data structures (hash maps, trees, graphs) and object-oriented design principles. The project aims to demonstrate a working understanding of how real-world VCS tools manage data integrity, history, and collaboration under the hood.

- **Milestone 1 — Object Model & Storage (Week 1–2):** Design core classes (Blob, Tree, Commit) and implement SHA-1/SHA-256 based content-addressable object storage. *(Deliverable: working init, add, and commit commands with objects persisted to disk.)*
- **Milestone 2 — History & Navigation (Week 3):** Implement commit history traversal (DAG-based, supporting linear commits) and the log command to display commit chronology with parent linkage.
- **Milestone 3 — Branching (Week 4):** Implement branch and checkout functionality, allowing multiple divergent commit histories and switching between them via reference pointers (HEAD).
- **Milestone 4 — Diff Engine (Week 5):** Implement a line-based diff algorithm (LCS/Myers) to compare file versions across commits. *(Deliverable: diff command showing additions/deletions between two commits.)*
- **Milestone 5 — Merge & Conflict Resolution (Week 6):** Implement three-way merge using lowest common ancestor (LCA) detection in the commit graph, with conflict marking for overlapping changes.
- **Milestone 6 — Polish & Evaluation (Week 7–8):** Build a CLI interface (and optional commit-graph visualization), benchmark diff/merge performance, and document design decisions, limitations, and comparisons with real Git in the final report.

Final Deliverable: A functional CLI-based Mini-Git tool supporting `init`, `add`, `commit`, `log`, `branch`, `checkout`, `diff`, and `merge` — backed by a complete technical report.

PROJECT APPROACH

Modern version control systems like Git are used daily, yet their internal mechanics—content-addressable storage, commit DAGs, and diffing—remain a black box to most developers. This project builds a simplified Version Control System (Mini-VCS) from scratch to replicate Git's core internals, demonstrating practical application of trees, graphs, hashing, and dynamic programming.

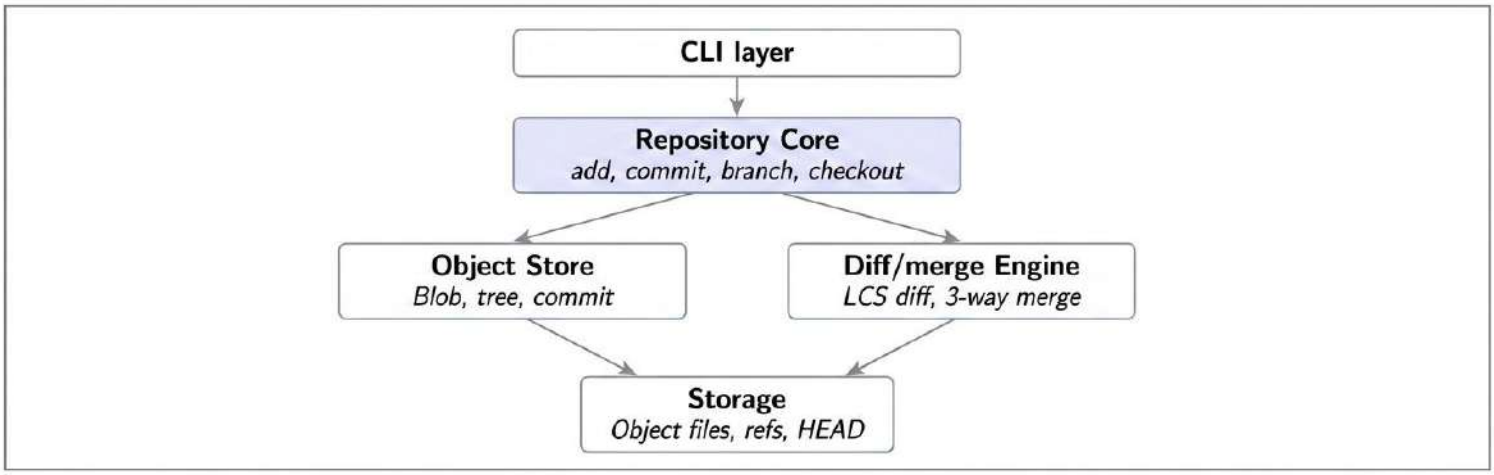
Design Approach: The system follows Git's object model: Blobs (file content), Trees (directory snapshots), and Commits (metadata + parent pointers), all addressed via SHA-1 hashes for automatic deduplication. The commit history is modeled as a Directed Acyclic Graph (DAG) rather than a simple tree, since merge commits have multiple parents—requiring graph traversal (BFS/DFS) for history and Lowest Common Ancestor (LCA) computation for merges.

Core Algorithmic Components:

• Content hashing for object storage and integrity • LCS-based/Myers diff algorithm (dynamic programming) for line-level file comparison • Three-way merge using LCA + diff, with conflict detection modeled on Git's `<<<<<<<` / `=======` / `>>>>>>>` markers • Object-oriented design uses a Repository class orchestrating Blob, Tree, Commit, and Branch classes, with the Strategy pattern applied to pluggable diff/merge algorithms.

System Architecture (High Level Diagram)(2 pts)

(Provide an overview of the system, identifying its main components and interfaces in the form of a diagram using a tool
of your choice).



Platforms & Technologies:

- **Language:** Java (strong OOP support, built-in MessageDigest for hashing, clean file I/O)
- **Storage:** Local filesystem, mimicking Git's `.git/objects` structure (hash-prefixed directories)
- **CLI:** Custom command-line interface (`init`, `add`, `commit`, `branch`, `merge`, `diff`, `log`)
- **Visualization:** Graphviz or a lightweight web UI (HTML/JS) to render the commit DAG
- **Version Control for Project:** Git/GitHub (used ironically to build our own VCS)
- **Testing:** JUnit for algorithm correctness (diff/merge edge cases)

Deliverable: A working CLI tool plus a technical report comparing our implementation against real Git's design tradeoffs (e.g., no packfiles/delta compression).

PROJECT OUTCOME

A fully functional command-line version control system replicating Git's core internals — content-addressable storage, commit history as a DAG, and a working diff/merge engine. Demonstrates practical mastery of hash tables, trees, graphs, and dynamic programming (Myers/LCS diff) applied to a real-world system problem, along with OOP design principles (composition, Strategy pattern for merge/diff strategies).

Deliverables:

- Working CLI tool supporting `init`, `add`, `commit`, `branch`, `checkout`, `log`, `diff`, `merge`
- Source code with modular class design (Blob, Tree, Commit, Repository)
- Commit graph visualizer (DAG rendering via Graphviz/D3.js)
- Technical report — architecture, algorithm complexity analysis (diff: O(ND)), comparison with real Git internals, limitations
- Demo — live walkthrough of branching, merging, and conflict resolution on a sample repository
- Test suite validating hashing correctness, merge conflict detection, and diff accuracy against known cases

ASSUMPTIONS

- Single-user, local repository only — no remote push/pull, networking, or multi-user collaboration.
- File-level tracking only; no support for binary diffing (text files assumed for diff/merge).
- SHA-1 used for hashing, acknowledging it's cryptographically weak but sufficient for demonstrating content-addressing (real Git has since moved toward SHA-256).
- No packfiles or delta compression — objects stored uncompressed for simplicity.
- Merge conflict resolution is manual (user edits conflict markers), not automatic.
- Filesystem is stable during operations (no concurrent access/locking handled).
- Command-line interface only; no GUI required for core functionality.

REFERENCES

- Pro Git Book (official, free) — https://git-scm.com/book/en/v2
- Git Internals chapter specifically — https://git-scm.com/book/en/v2/Git-Internals-Plumbing-and-Porcelain
- Git Objects — https://git-scm.com/book/en/v2/Git-Internals-Git-Objects
- Git official documentation — https://git-scm.com/docs
- "Git from the Bottom Up" by John Wiegley — https://jwiegley.github.io/git-from-the-bottom-up/

- Git source code (GitHub mirror) — https://github.com/git/git